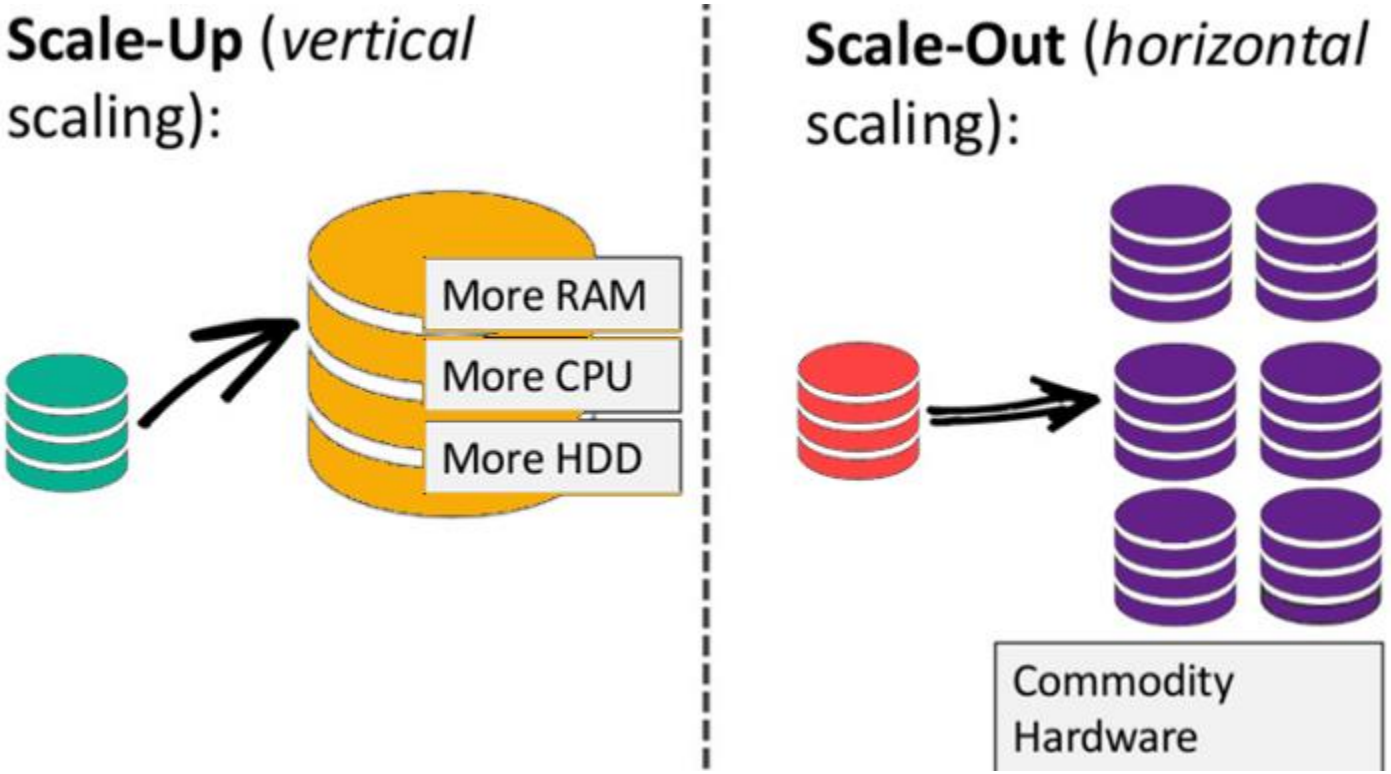
RDBMS vs NoSQL

Course 4

(Traditional) Workloads

- OLTP:
  - The most used (at least at school)
  - Based on transactions
  - Known access patterns with predefined schema
  - Write-heavy
- OLAP:
  - Used for data warehouses
  - Access patterns not known entirely (Ad-hoc)
  - Read heavy
  - Large data volumes
  - Possibly long operation (minutes, hours)

Scaling Strategies



RDBMS vs NoSQL

- typical story cycle in IT: every new technology destroys and replaces an older one; PCs replaced mini-computers. Smartphones unseated cameras and flip phones (they are coming back :-D). Online streaming wipes out video rental and music CDs.
- Are big data technologies wiping out relational database management systems?

NoSQL

- Stands for Not Only SQL
- Non-relational data storage systems
- Usually do not require a fixed table schema
- The concept of joins is eliminated
- All NoSQL offerings relax one or more of the ACID properties
- Meta S. Brown: <<if you need to get the data to deliver precise answers, then you've got to use a relational database. If you need an approximate answer in a big hurry, then a NoSQL database is the way to go. >>

NoSQL – The Beginning

- Three major papers were the seeds of the NoSQL movement
  - BigTable on top of GFS(Google)
  - Dynamo (Amazon)
    - Gossip protocol (discovery and error detection)
    - Distributed key-value data store
    - Eventual consistency
  - CAP Theorem

GFS

Google has reexamined traditional choices /assumptions and explored radically different points in the design space:

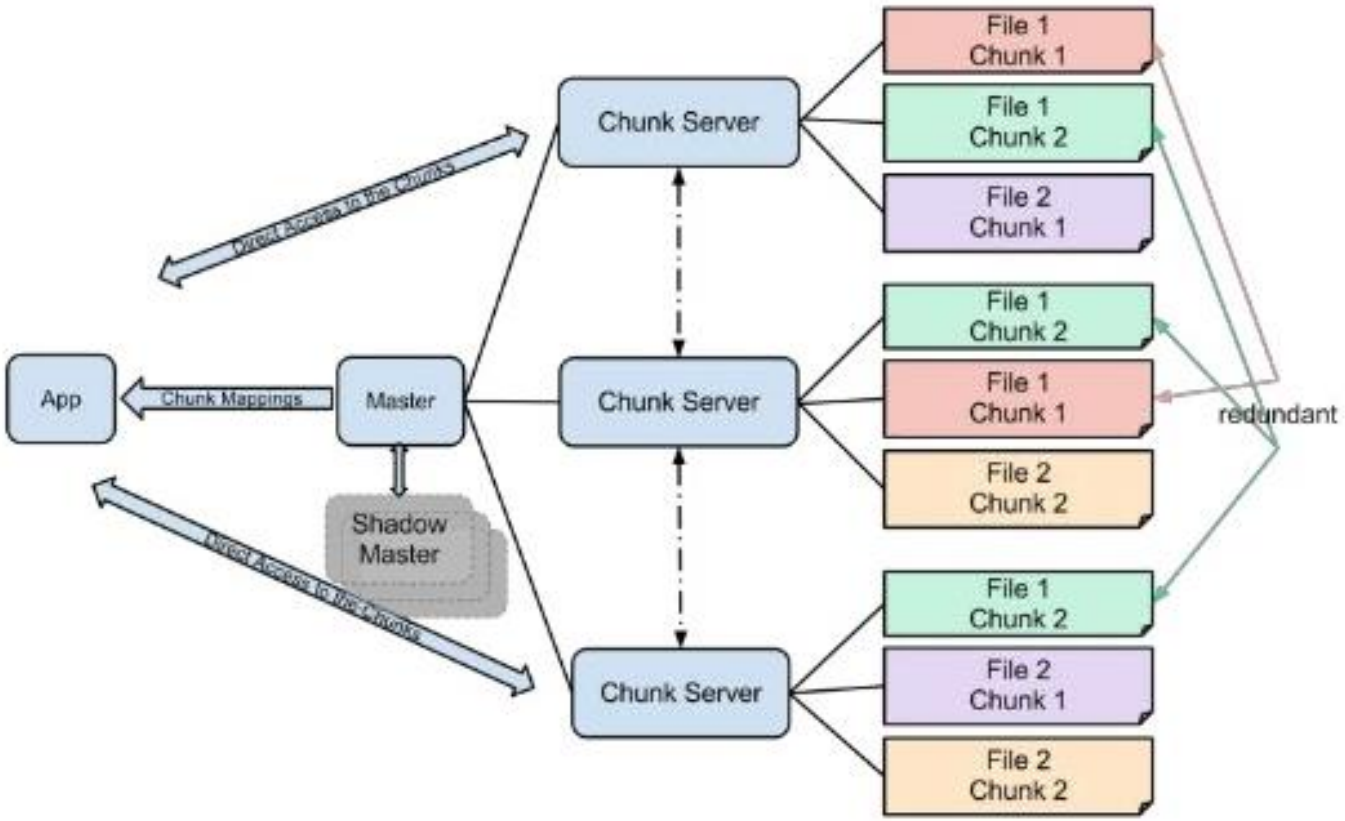
1. First, **component failures** are the norm rather than the exception
   - The system is built from many inexpensive commodity components that often fail. It must constantly monitor itself and detect, tolerate, and recover promptly from component failures on a routine basis
2. Second, **files are huge** by traditional standards. Multi-GB files are common
3. Third, most files are **mutated by appending new data** rather than overwriting existing data
4. Fourth, **co-designing the applications and the file system API** benefits the overall system by increasing flexibility

GFS Architecture

- GFS is a cluster of computers. A cluster is simply a network of computers. Each cluster might contain hundreds or even thousands of machines. In each GFS cluster there are three main entities:

1. Clients
2. Master servers
3. Chunk servers.

GFS Architecture



Source: wikipedia

GFS Architecture

- Master Server is the coordinator for the cluster. Its task include:

1. Maintaining an operation log, that keeps track of the activities of the cluster. The operation log helps keep service interruptions to a minimum if the master server crashes, a replacement server that has monitored the operation log can take its place.
2. The master server also keeps track of metadata, which is the information that describes chunks. The metadata tells the master server to which files the chunks belong and where they fit within the overall file.

GFS Architecture

- Client can be other computers or computer applications that make a file request. Requests can range from retrieving and manipulating existing files to creating new files on the system.
- Chunk Servers store 64-MB file chunks. The chunk servers don't send chunks to the master server. Instead, they send requested chunks directly to the client. The GFS copies every chunk multiple times and stores it on different chunk server. Each copy is called a replica. By default, the GFS makes three replicas per chunk, but users can change the setting and make more or fewer replicas if desired.

Big Table

- Is a large scale, fault tolerant, self managing system that includes terabytes of memory and petabytes of storage. It can handle millions of reads/writes per second.
- Is a distributed hash mechanism built on top of GFS. It is not a relational database. It doesn't support joins or SQL type queries.
- Provides lookup mechanism to access structured data by key.
- Machines can be added and deleted while the system is running and the whole system just works.
- Each data item is stored in a cell which can be accessed using a row key, column key, or timestamp (**wide column store**).
- Has three different types of servers: (Master, Tablet ,Lock Servers)

Big Table

- Is a sparsely populated table that can scale to billions of rows and thousands of columns, allowing you to store terabytes or even petabytes of data.
- A single value in each row is indexed; this value is known as the row key.
- Is ideal for storing very large amounts of single-keyed data with very low latency.
- It supports high read and write throughput at low latency

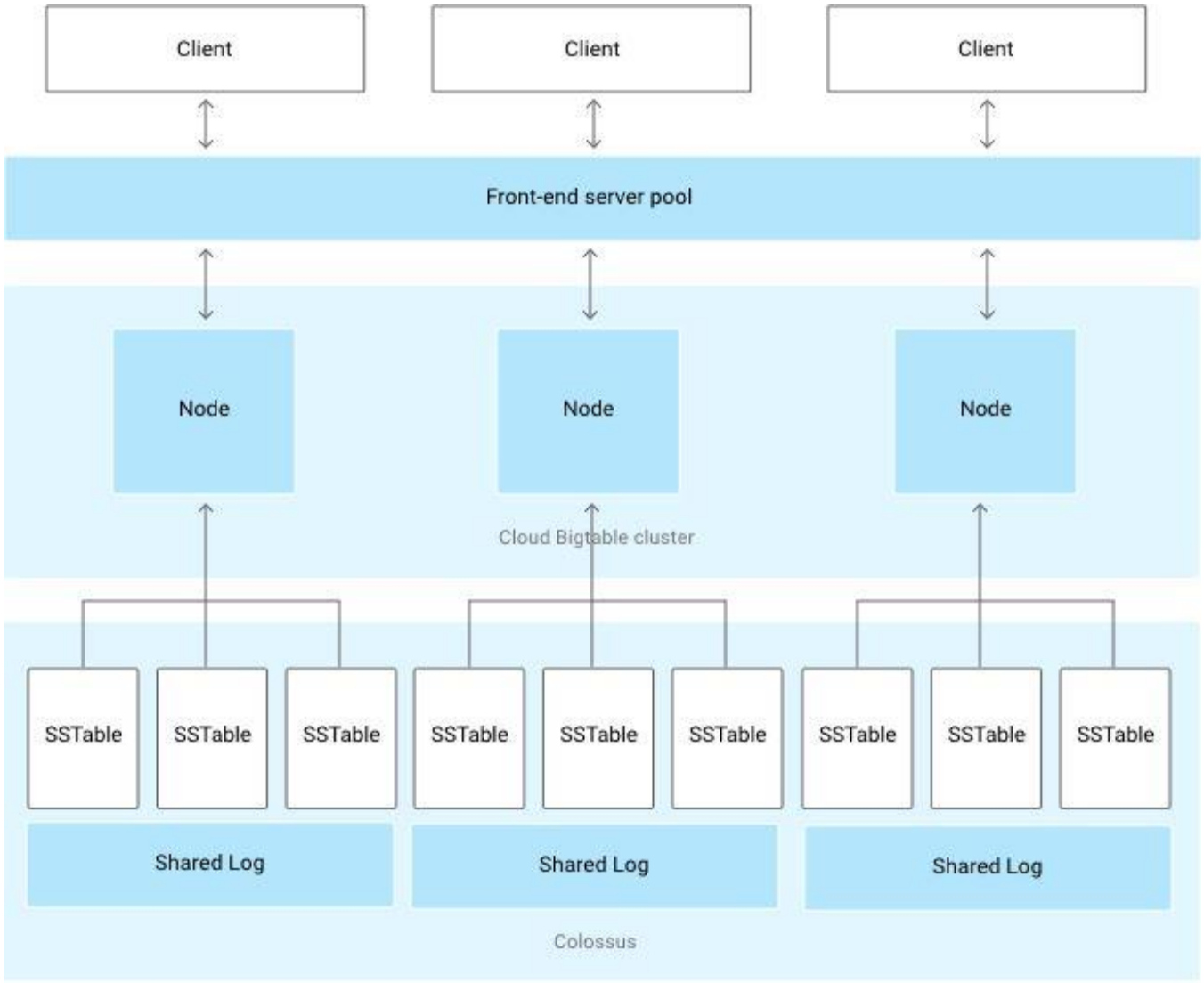
Big Table

Row Key	Column Family: profile	Column Family: activity
user001	name: John email: john@example.com	last_login: 2025-10-19 pages_viewed: 12
user002	name: Alice email: alice@example.com	last_login: 2025-10-18 pages_viewed: 8
user003	name: Bob email: bob@example.com	last_login: 2025-10-20 pages_viewed: 15
user004	name: Carol email: carol@example.com	last_login: 2025-10-17 pages_viewed: 5
user005	name: Dave email: dave@example.com	last_login: 2025-10-20 pages_viewed: 20

Big Table

- Stores data in massively scalable tables, each of which is a sorted key/value map. The table is composed of rows, each of which typically describes a single entity, and columns, which contain individual values for each row.
- Each row is indexed by a single row key, and columns that are related to one another are typically grouped together into a column family. Each column is identified by a combination of the column family and a column qualifier, which is a unique name within the column family.
- Each row/column intersection can contain multiple cells at different timestamps, providing a record of how the stored data has been altered over time. Bigtable tables are sparse; if a cell does not contain any data, it does not take up any space.

Big Table



Tablet servers

Tablets (blocks of rows)

Filesystem

Source: cloud.google.com/bigtable/

CAP Theorem

- Three properties of a system: consistency, availability and partitions
- You can have at most two of these three properties for any shared-data system
- To scale out, you have to partition. That leaves either consistency or availability to choose from
  - In almost all cases, probably is best to choose availability over consistency

Availability

- Traditionally, thought of as the server/process available five 9's (99.999 %).
- However, for large node system, at almost any point in time there's a good chance that a node is either down or there is a network disruption among the nodes.
  - Want a system that is resilient in the face of network disruption

Consistency

- A consistency model determines rules for visibility and apparent order of updates.
- For example:
  - Row R is replicated on nodes N1 and N2
  - Client C1 writes row R to node N2
  - Some period of time t elapses
  - Client C2 reads row R from node N1
  - Does client C2 see the write from client C1?
  - Consistency is a continuum with tradeoffs
  - For NoSQL, the answer would be: **maybe**
  - CAP Theorem states: Strict Consistency can't be achieved at the same time as availability and partition-tolerance.

ACID vs BASE

- While RDBMS uses ACID (Atomicity, Consistency, Isolation, Durability) as a mechanism for ensuring the consistency of data, non-relational DBMS use BASE.
- BASE stands for Basically Available, Soft state, and Eventual Consistency. Of these, eventual consistency is most important because it is responsible for conflict resolution when data is in motion between nodes in a distributed implementation.
- The data state is maintained by the software and the access model relies on basic availability.

Eventual Consistency

- When no updates occur for a long period of time, eventually all updates will propagate through the system and all the nodes will be consistent
- For a given accepted update and a given node, eventually either the update reaches the node or the node is removed from service

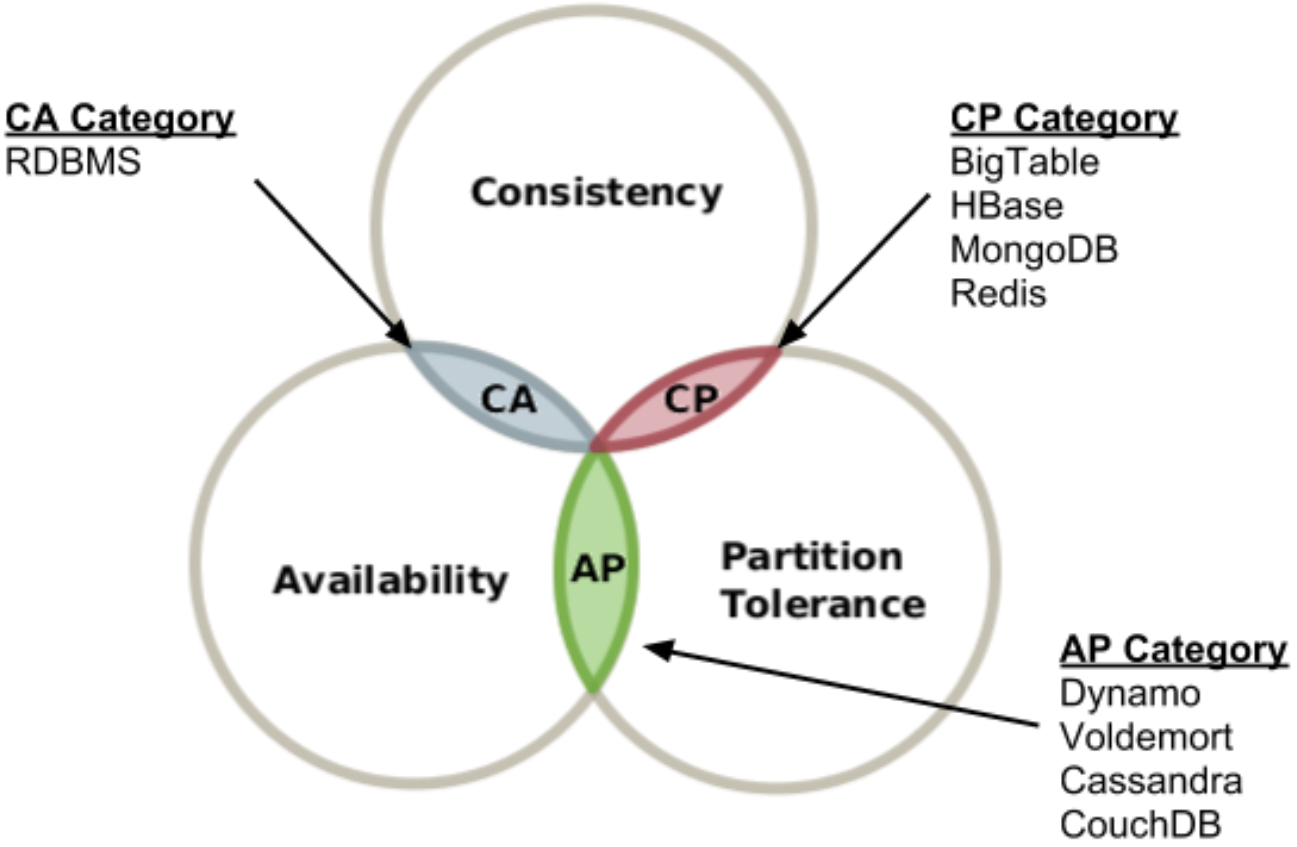
ACID vs BASE

- ACID:
  - Strong consistency.
  - Less availability.
  - Pessimistic concurrency.
  - Complex.
- BASE:
  - Availability is the most important thing (CAP).
  - Weaker consistency (Eventual).
  - Best effort.
  - Simple and fast.
  - Optimistic.

CAP

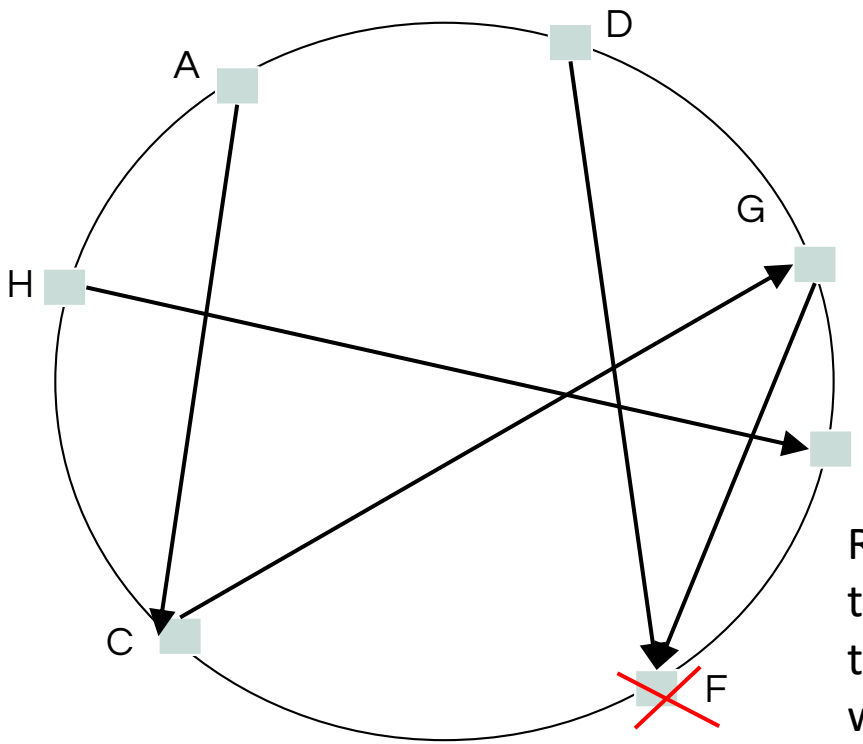
- **C**onsistency: Clients should read the same data. There are many levels of consistency.
  - Strict Consistency – RDBMS.
  - Tunable Consistency – Cassandra.
  - Eventual Consistency – Amazon Dynamo.
- **A**vailability: Data to be available.
- **P**artition Tolerance: Data to be partitioned across network segments due to network failures.

CAP


CA Category
RDBMS
Consistency
CP Category
BigTable
HBase
MongoDB
Redis
CA
CP
Availability
AP
Partition
Tolerance
AP Category
Dynamo
Voldemort
Cassandra
CouchDB

Gossip Protocol

One of the preferred communication protocols in a distributed environment is Gossip Protocol.

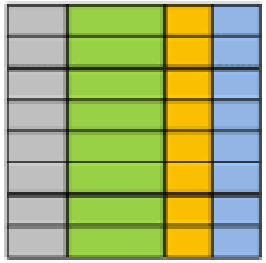


- All the nodes talk to each other peer wise.
- There is no global state.
- No single point of coordination.
- If one node goes down and there is a Quorum, load for that node is shared among others.
- Self managing system.
- If a new node joins, load is also distributed.

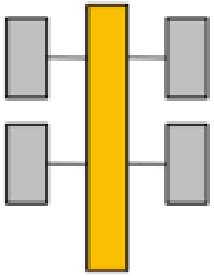
Requests coming to F will be handled by the nodes who take the load of F, let's say C with the **hint** that it took the requests which were for F, when F becomes available, F will get this Information from C. Self healing property.

Storage Models

Relational



Analytical (OLAP)



Column-Family



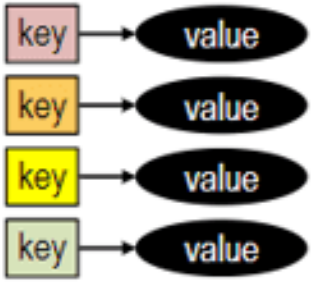
Graph



Document



Key-Value



Storage Models

- Key-value stores:
  - Processing a constant stream of small reads and writes.
- Document databases:
  - Natural data modeling. Programmer friendly. Rapid development. Web friendly, CRUD.
- *RDMBS*:
  - OLTP. SQL. Transactions. Relations.
- Columnar:
  - Handles size well. Massive write loads.
  - High availability. Multiple-data centers.
  - MapReduce
- Graph:
  - Graph algorithms and relations.

Key-Value

- The database stores data as a collection of key/value pairs. This is a simple method of storing data, and it is known to scale well.
- The key-value pair is a well established concept in many programming languages, as an associative array or data structure. A **key-value is** also commonly referred to as a **dictionary or hash**.
- The key in a key-value pair must be unique. This is the unique identifier that allows you to access the value associated with that key.

Key-Value

- The value in a key-value store can be anything, from text (long or short), a number, markup code such as HTML, programming code, an image, etc.
- The value could also be a list, or even another key-value pair encapsulated in an object.
- Some key-store DBMSs allow you to specify a data type for the value. For example, you could specify that the value should be an integer. Other DBMSs don't provide this functionality so, the value could be of any type.
- They provide **simple operations: get, put and delete**. The data querying has to be handled at the application level.

Key-Value: example

- DynamoDB:
  - stores data as items in tables, where each item is uniquely identified by a primary key. This key can be:
    - Partition Key (Hash Key): A single attribute that uniquely identifies each item.
    - Composite Key (Partition Key + Sort Key): Allows multiple items with the same partition key but different sort keys, enabling range queries.
  - Each item is essentially a key-value pair, where:
    - The key is the primary key.
    - The value is a flexible set of attributes (like a JSON document).

Partition Key (`UserID`)	Attributes (Value)
`user123`	`{ "theme": "dark", "language": "en" }`
`user456`	`{ "theme": "light", "language": "fr" }`
`user789`	`{ "theme": "dark", "language": "es" }`

Key-Value

Key Features That Enhance Key-Value Behavior:

- **Flexible schema**: Each item can have different attributes. No need to define columns ahead of time.
- **Fast lookups**: Retrieve items directly using the key with single-digit millisecond latency.
- **Secondary indexes**: You can define alternate keys (Global or Local Secondary Indexes) to query by other attributes.
- **Conditional writes**: Ensures atomic updates and prevents race conditions.
- **TTL** (Time to Live): Automatically delete items after a set time, useful for caching.

Document

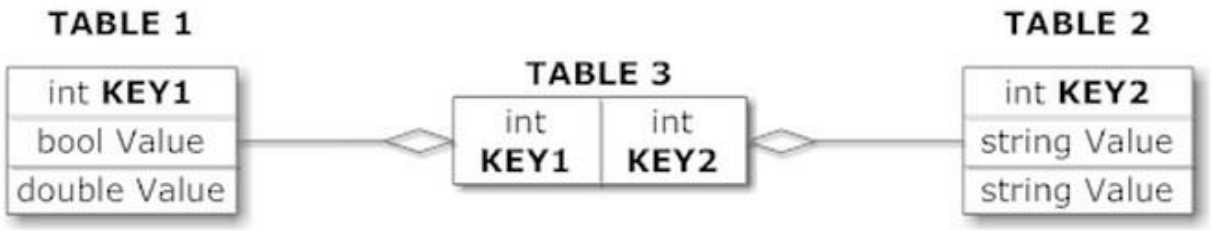
- A document-oriented database is a designed for storing, retrieving, and managing document-oriented, or **semi structured data**. The concept of a document-oriented database is the notion of a Document. While each document-oriented database implementation differs on the details of this definition, in general, they all assume documents encapsulate and encode data (or information) in some standard format or encoding. Encodings used include XML, YAML, JSON and BSON, as well as binary forms like PDF and Microsoft Office documents

Document

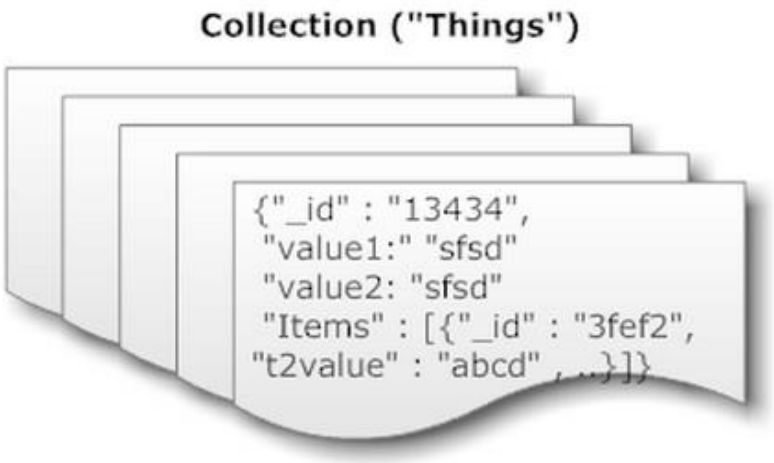
- Document dbs are a type of key-value db: each document has a unique identifier (the key) and the document itself serves as the value.
- The difference between these two models is that:
  - in a key-value database, the data (value) is treated as opaque and the database doesn't know about the data held within it; it's up to the application to understand what data is stored
  - in a document store each document contains some kind of metadata (a degree of structure to the data). They often come with an API or query language that allows users to retrieve documents based on the metadata they contain. They also allow for complex data structures, as you can nest documents within other documents.

Document

Relational Model

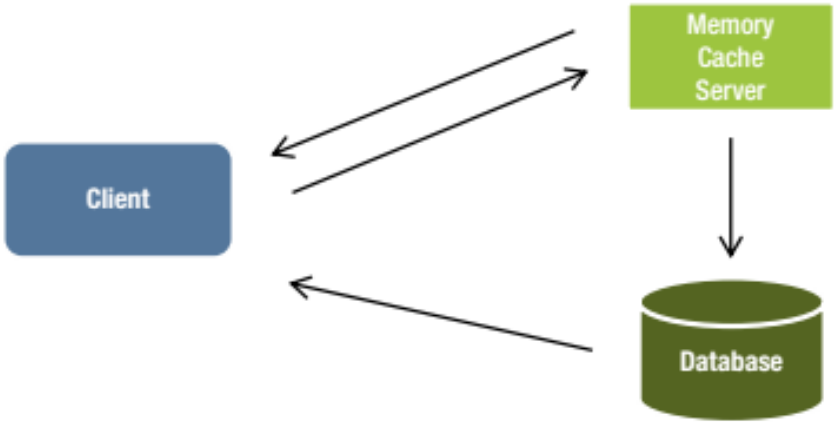


Document Model



source: www.dzone.com

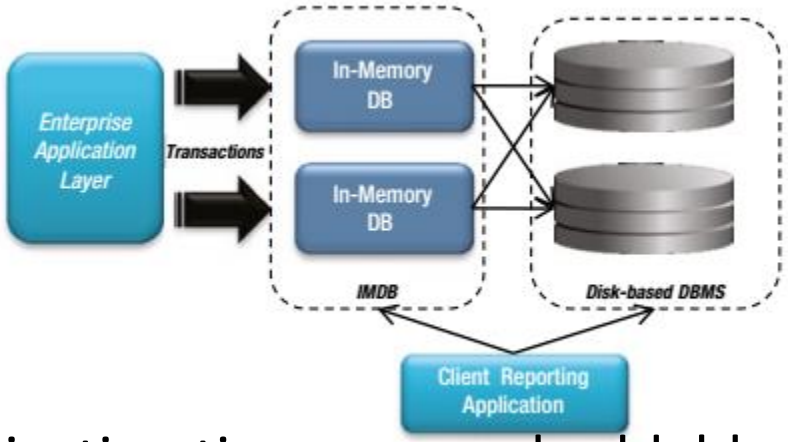
Document - In Memory Caching



- In-Memory Caching
  - Data is deployed between the application and the database to reduce database load
  - Ideal for caching data to memory that is repeatedly accessed.
  - Data is not replicated or persisted across servers.
  - It harnesses the aggregate memory of many distributed machines by using a hashing algorithm.

Facebook uses 800 servers for over 28 TBs of memory EHCache.

Document - In Memory Database



- In Memory Database
  - Data is deployed in the application tier as an embeddable database—for example, Derby
  - Reading and writing data is as fast as accessing RAM. For example, with a 1.8-GHz processor, a read transaction can take less than 5 microseconds, with a write transaction taking less than 15 microseconds.
  - The database fits entirely in physical memory.
  - The data is managed in memory with optimized access algorithms.
  - Transaction logs and database checkpoint files stored to disk.

Document: examples

MongoDB:

- composed of databases containing “collections”. A collection is composed of “documents” and each document is composed of fields.
- index a collection to increase the performance of data lookup. MongoDB returns something called a “cursor” which serves as a pointer to the data.
- offers the option of counting or classifying the data without extracting it. MongoDB supports BSON, the binary implementation of JSON.

Document

CouchDB:

- databases composed of documents consisting of fields and attachments as well as a “description” of the document in the form of metadata that is automatically maintained by the system.
- The underlying technology features all ACID capabilities from the RDBMS world.
- The advantage in CouchDB over relational is that the data is packaged and ready for manipulation or storage rather than scattered across rows and tables.

Columnar

- Relational databases are row oriented, the data in each row of a table is stored together. In a columnar, or column-oriented database, the data is stored across rows. It is very easy to add columns, and they may be added row by row, offering great flexibility, performance, and scalability. When you have volume and variety of data, you might want to use a columnar database. It is very adaptable; you simply continue to add columns.
- They use traditional database languages like SQL to load data and perform queries. The database can more precisely access the data it needs for a query rather than scanning and discarding unwanted data in rows. Query performance is often increased, particularly in very large data sets.

Columnar

There are 2 (somehow similar) types:

- Column stores

Stores data tables by column rather than by row
Serializes all of the values of a column together, similar to an index organization for row oriented dbs
Reduces the amount of data read from disk by compressing the similar columnar data and by reading only the necessary data
Examples

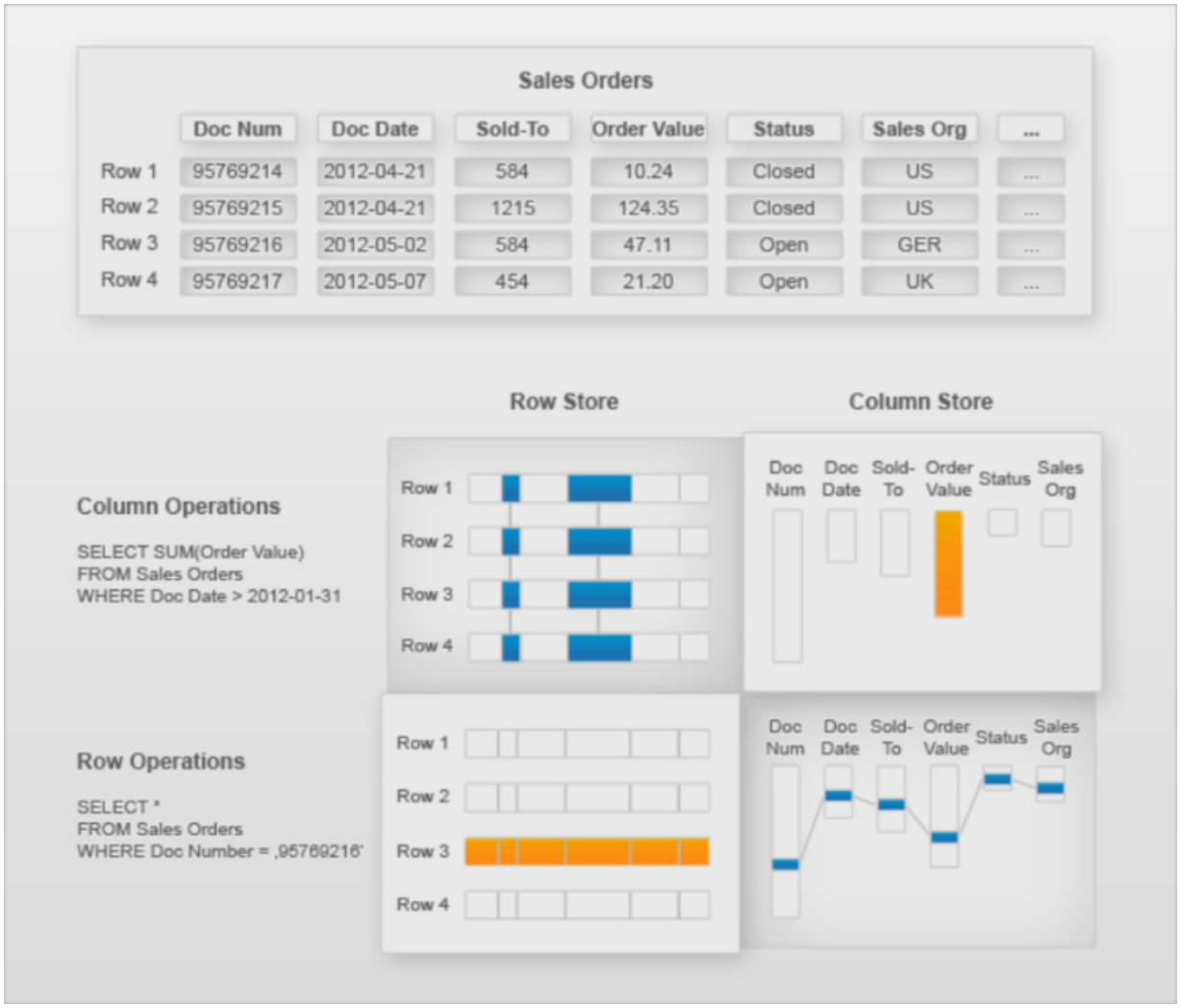
- Apache Parquet, Amazon Redshift
- MariaDB ColumnStore

Columnar



source: http://www.data-

Columnar



source: http://www.data-design.org

Columnar

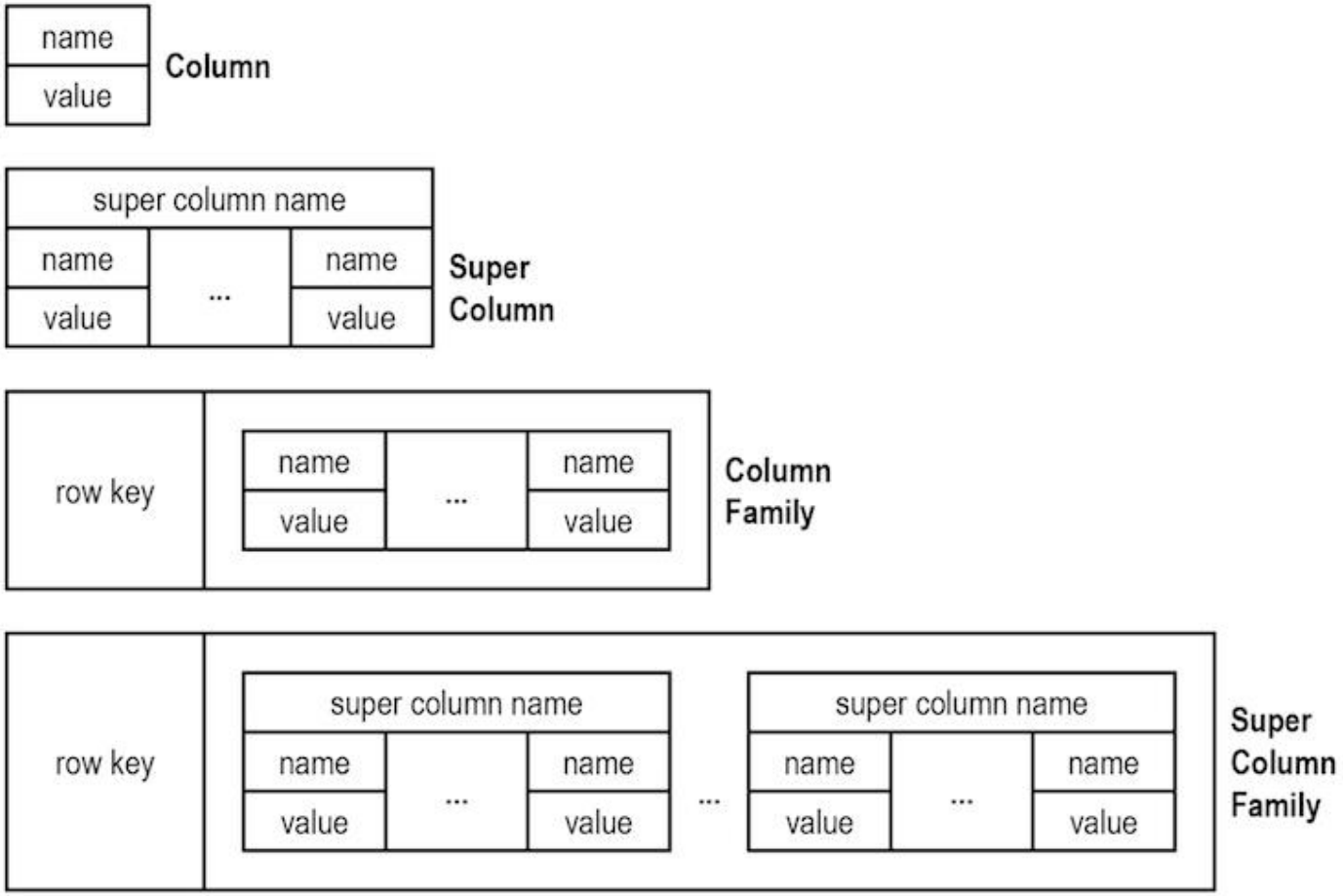
- Wide column databases

Has an architecture that uses persistent, sparse matrix, multi-dimensional mapping (row-value, column-value, and timestamp) in a tabular format meant for massive scalability (over and above the petabyte scale)

Examples:

- BigTable, Cassandra, HBase

Columnar


name
value
Column
super column name
name
value
...
name
value
Super Column
row key
name
value
...
name
value
Column Family
row key
super column name
name
value
...
name
value
...
super column name
name
value
...
name
value
Super Column Family

Columnar: example

- Hbase: the design of HBase is modeled on Google's BigTable. Implementations of HBase are highly scalable, sparse, distributed, persistent multidimensional sorted maps. The map is indexed by a row key, column key, and a timestamp; each value in the map is an uninterpreted array of bytes. When big data implementation requires random, real-time read/write data access, HBase is a very good solution. It is often used to store results for later analytical processing.

Columnar

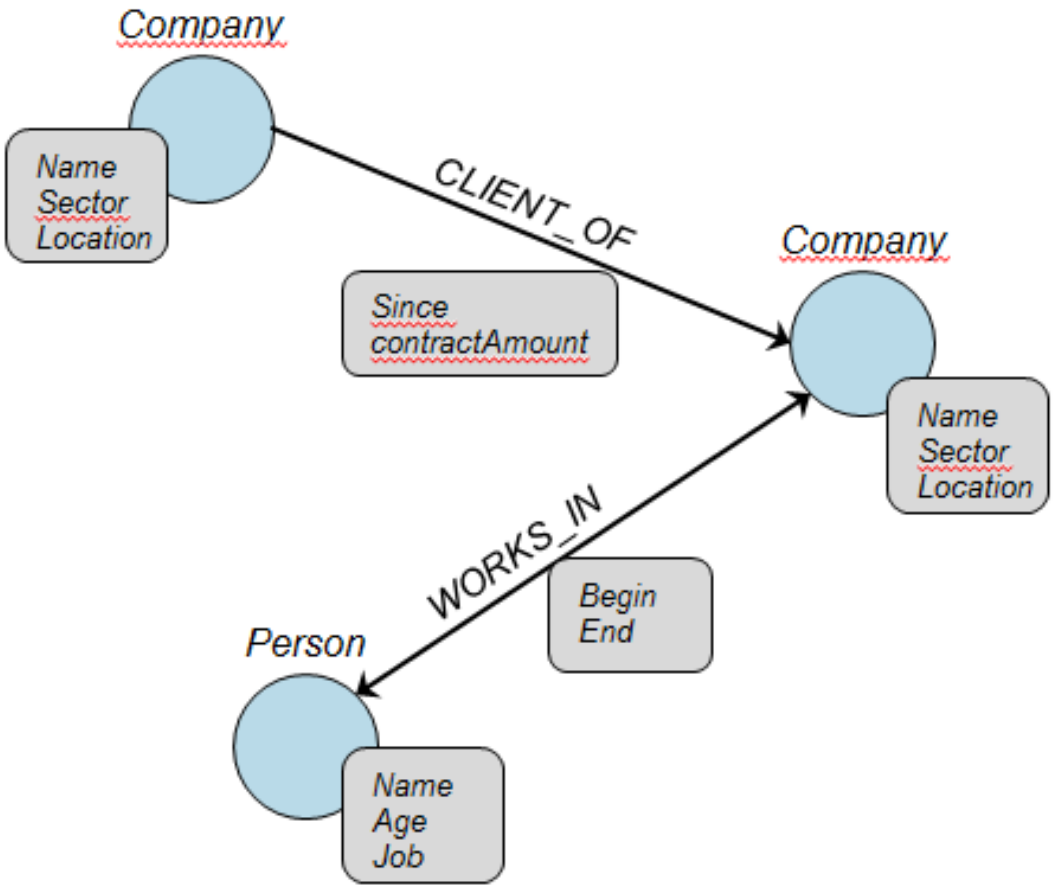
Similarities:

- Column-oriented data access: Both store and access data by columns rather than rows, which can improve performance for certain queries.
- Efficient for sparse data: They handle sparse datasets well, storing only non-null values.
- Good for analytics: Column-based access is ideal for aggregations and filtering on specific fields.
- Compression-friendly: Columnar formats allow better compression since similar data types are stored together.

Graph

- The fundamental structure for graph databases is called "**node-relationship**". This structure is most useful when you must deal with highly interconnected data. Nodes and relationships support properties, a key-value pair where the data is stored. These databases are navigated by following the relationships. This kind of storage and navigation is not possible in RDBMSs due to the rigid table structures and the inability to follow connections between the data wherever they might lead us. A graph database might be used to manage geographic data for oil exploration or to model and optimize a telecommunications provider's networks.

Graph



source: blog.octo.com

Graph: example

- Neo4J: is an ACID transaction database offering high availability through clustering. It is a trustworthy and scalable database that is easy to model because of the node-relationship properties' fundamental structure and how naturally it maps to our own human relationships. It does not require a schema, nor does it require data typing, so it is inherently very flexible.

Graph Connected Data

Property Graphs and RDF (Resource Description Framework)

Think of this as the NoSQL of graphs — pragmatic, developer-friendly, and built for speed.
Nodes: Entities (e.g., Person, Product)
Edges: Relationships (e.g., KNOWS, BOUGHT)
Properties: Key-value pairs on both nodes and edges

the semantic web's native language — formal, standardized, and built for interoperability.
Everything is a triple: **Subject** → **Predicate** → **Object** (e.g., :Alice :knows :Bob)
No properties on edges — instead, you reify relationships (i.e., turn them into nodes)

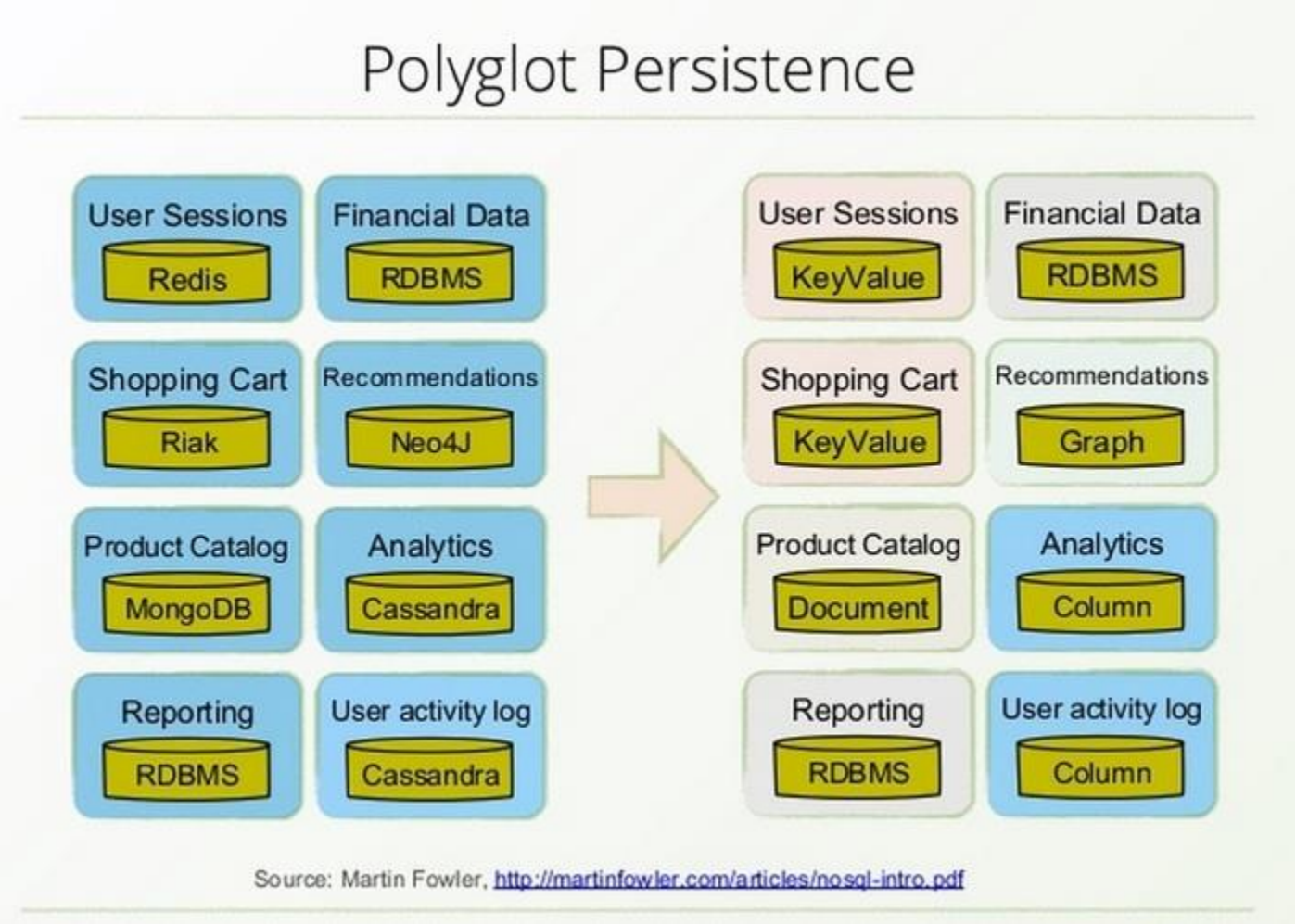
Graph Connected Data

Feature	Property Graph	RDF
Schema	Optional (schema-less or enforced)	Schema via RDFS
Flexibility	High — great for agile modeling	High, but more formal and verbose
Semantics	Implicit (developer-defined)	Explicit (machine-readable ontologies)
Query language	Cypher, Gremlin, PGQL MATCH (a:Person)-[:KNOWS]->(b:Person) RETURN b	SPARQL SELECT ?b WHERE { :Alice :knows ?b }
Use case	Social networks, fraud detection, recommendations	Knowledge graphs, data integration, linked data, ontologies

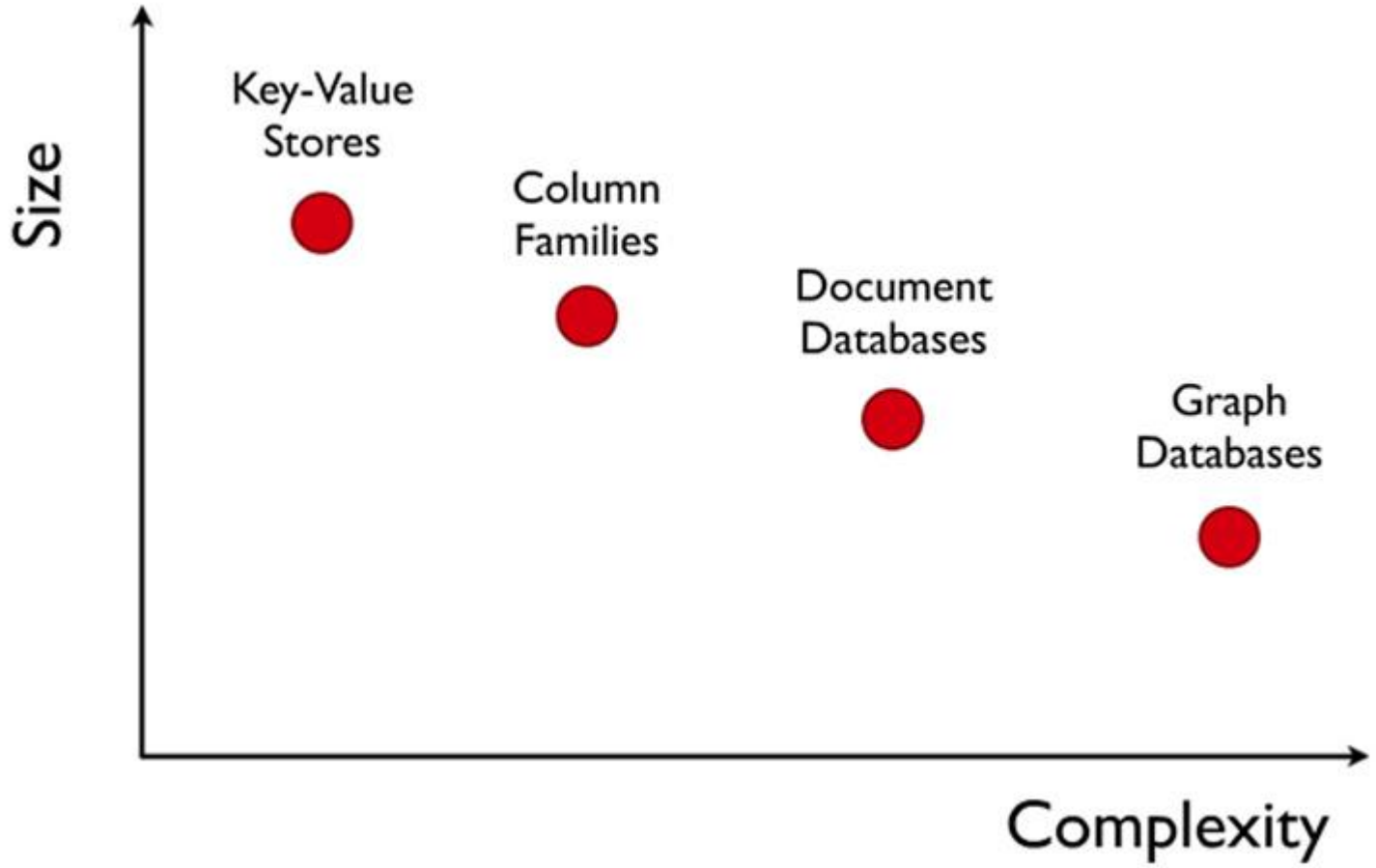
Polyglot

- The official definition of polyglot is “someone who speaks or writes several languages”. The term is redefined as a set of applications that use several core database technologies, the most likely outcome of a big data implementation planning. It is difficult to choose one persistence style no matter how narrow the approach to big data might be. A polyglot persistence database is used when it is necessary to solve a complex problem by breaking that problem into segments and applying different database models. It is then necessary to aggregate the results into a hybrid data storage and analysis solution.

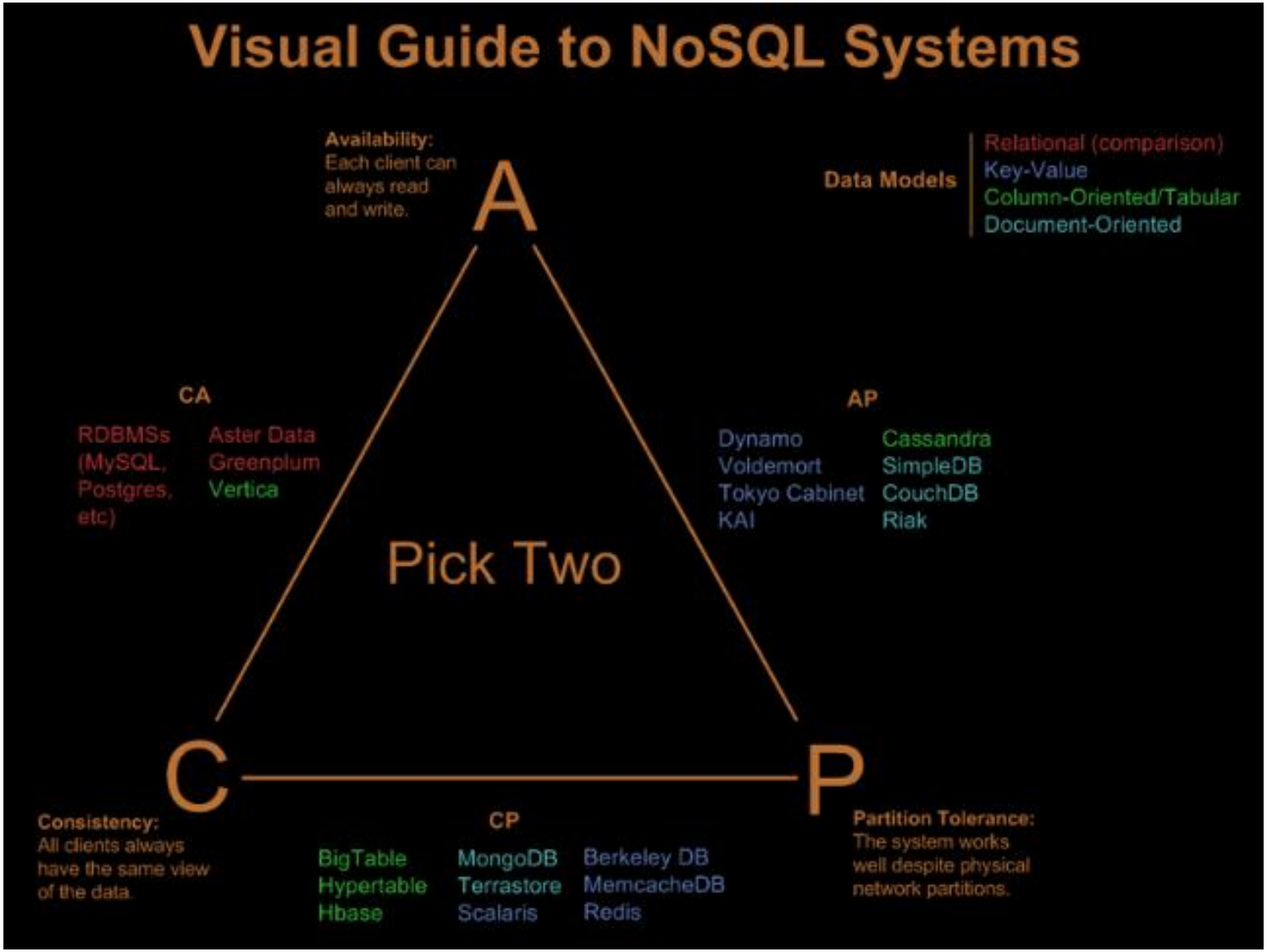
Polyglot


Polyglot Persistence
User Sessions
Redis
Financial Data
RDBMS
Shopping Cart
Riak
Recommendations
Neo4J
Product Catalog
MongoDB
Analytics
Cassandra
Reporting
RDBMS
User activity log
Cassandra
User Sessions
KeyValue
Financial Data
RDBMS
Shopping Cart
KeyValue
Recommendations
Graph
Product Catalog
Document
Analytics
Column
Reporting
RDBMS
User activity log
Column
Source: Martin Fowler, http://martinfowler.com/articles/nosql-intro.pdf

Size vs Complexity



CAP Compliant



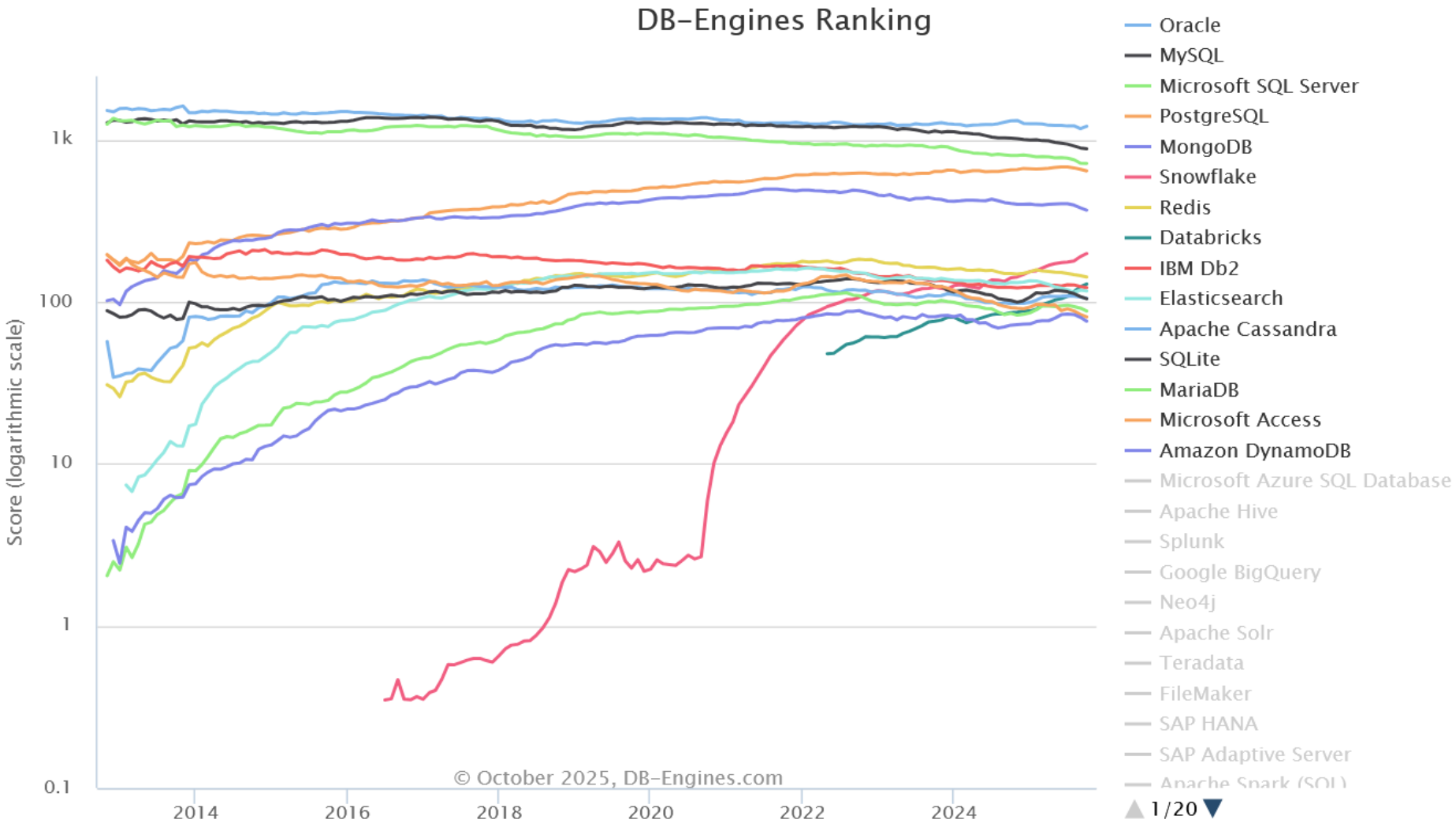
Databases Ranking

425 systems in ranking, October 2025

Rank			DBMS	Database Model	Score		
Oct 2025	Sep 2025	Oct 2024			Oct 2025	Sep 2025	Oct 2024
1.	1.	1.	Oracle	Relational, Multi-model	1212.77	+42.15	-96.67
2.	2.	2.	MySQL	Relational, Multi-model	879.66	-12.11	-143.09
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	715.05	-2.27	-87.04
4.	4.	4.	PostgreSQL	Relational, Multi-model	643.20	-13.97	-8.96
5.	5.	5.	MongoDB	Document, Multi-model	368.01	-12.49	-37.20
6.	6.	↑ 7.	Snowflake	Relational	198.65	+8.46	+58.05
7.	7.	↓ 6.	Redis	Key-value, Multi-model	142.33	-2.84	-7.30
8.	↑ 9.	↑ 14.	Databricks	Multi-model	128.80	+4.74	+43.21
9.	↓ 8.	9.	IBM Db2	Relational, Multi-model	122.37	-1.81	-0.40
10.	10.	↓ 8.	Elasticsearch	Multi-model	116.67	-1.59	-15.18
11.	↑ 12.	11.	Apache Cassandra	Wide column, Multi-model	105.16	-1.82	+7.56
12.	↓ 11.	↓ 10.	SQLite	Relational	104.56	-3.32	+2.64
13.	13.	↑ 15.	MariaDB	Relational, Multi-model	87.77	-3.69	+2.88
14.	14.	↓ 12.	Microsoft Access	Relational	80.79	-2.82	-11.36
15.	15.	↑ 17.	Amazon DynamoDB	Multi-model	75.91	-4.37	+4.06
16.	16.	16.	Microsoft Azure SQL Database	Relational, Multi-model	75.49	-3.70	+0.95
17.	17.	↑ 18.	Apache Hive	Relational	75.22	-0.88	+22.65
18.	18.	↓ 13.	Splunk	Search engine	73.87	-1.89	-17.40
19.	19.	19.	Google BigQuery	Relational	63.20	-2.80	+12.02
20.	20.	↑ 21.	Neo4j	Graph	52.51	-1.27	+10.01

Source: http://db-engines.com/en/ranking

Databases Ranking



Source: http://db-engines.com/en/ranking